

PARTIU MARROCOS / KEROMINDPARTIU MARROCOS / KEROMIND

PM-NIGHT-RUN-02 — Documento Mestre Cumulativo

Documento Mestre Cumulativo

15 de setembro de 2026

Execução autônoma sequencial autorizada pelo fundador (comando PM-NIGHT-RUN-02, recebido logo após o fechamento de PM-NIGHT-RUN-01). Este documento **cresce progressivamente** — nunca é reescrito do zero, seções antigas nunca são apagadas, só atualizadas com status novo.

Ordem autorizada: Booking Foundation → Payment Foundation → Finance Core Real 01 → Travel Document Foundation → Trip Operation Foundation → Traveler Area V1 → Notifications Foundation → Post-Trip Foundation → PARAR (limite máximo desta autorização).

Estado de partida (confirmado por PM-NIGHT-RUN-01_FINAL, não presumido): 5 etapas anteriores GREEN, 359/359 testes, typecheck limpo, build limpo, 17 rotas, 31 migrations aplicadas, nenhuma migration destrutiva, Git ainda não inicializado, WABA ainda não homologado em conta real. Já existem: Tenant Core, RLS, Auth, RBAC, Gates, Audit, SecretProvider, Cost Control, Tool Broker, Job Engine, CRM, Attribution, i18n foundation, CRM Evolution, Lead Scoring, Next Best Action, Repescagem estruturada, Finance Core contracts, Proposal real/versionada, Gate COMERCIAL, aceite/recusa de proposta.

Índice de etapas

#	Etapas	Status	Relatório
1	Booking Foundation	GREEN	docs/ BOOKING_FOUNDATION_01_FECHAMENTO.md
2	Payment Foundation	GREEN	docs/ PAYMENT_FOUNDATION_01_FECHAMENTO.md
3	Finance Core Real 01	GREEN	docs/ FINANCE_CORE_REAL_01_FECHAMENTO.md
4	Travel Document Foundation	GREEN	docs/ TRAVEL_DOCUMENT_FOUNDATION_01_FECHAMENTO.md
5	Trip Operation Foundation	não iniciada	docs/ TRIP_OPERATION_FOUNDATION_01_FECHAMENTO.md
6	Traveler Area V1	não iniciada	docs/ TRAVELER_AREA_V1_FECHAMENTO.md
7	Notifications Foundation	não iniciada	docs/ NOTIFICATIONS_FOUNDATION_01_FECHAMENTO.md
8	Post-Trip Foundation	não iniciada	docs/ POST_TRIP_FOUNDATION_01_FECHAMENTO.md

Etapa 1 — Booking Foundation

Objetivo

Transformar uma Proposal ACEITA numa reserva/operação comercial real, com máquina de estados explícita e passageiros associados.

Arquitetura

Booking não duplica dados da Proposal (a Proposal ACEITA já é o snapshot imutável — só a FK **proposalId** é guardada; roteiro/preço/moeda/datas lidos via **booking.proposal.***). Idempotência estrutural: **@@unique([tenantId, proposalId])** — uma Proposal só gera UM Booking, garantido pelo banco. Máquina de estados explícita (**BookingStatus**: AGUARDANDO_PAGAMENTO → PAGAMENTO_PARCIAL/PAGO → AGUARDANDO_DOCUMENTOS/CONFIRMADA → EM_OPERACAO → CONCLUIDA, CANCELADA de qualquer estado não-terminal exceto EM_OPERACAO), tabela pura testável (mesmo padrão de **transicaoValida** em Gates). **Traveler** promovido a tabela real (consumidor real já autorizado na Etapa 4 desta mesma janela) — privacy-by-design: nenhum documento sensível/imagem, isso é Etapa 4.

Migrations

2 novas, aditivas: **20260915040000_booking_foundation_01** (tabelas), **20260915040100_enable_rls_booking_foundation_01** (RLS). **33 migrations no total.**

RBAC

2 permissões novas: **bookings.view/bookings.manage**. Administrador/Vendas ganham ambas; Atendimento só view.

Segurança

RLS testado (isolamento multi-tenant real, inclusive "Tenant B não enxerga Proposal de A por RLS, então nunca cria Booking indevido"). **BOOKING_CRIADO/BOOKING_STATUS_ALTERADO** auditados.

Testes

19 novos: 7 (**booking-transicoes.test.ts**, máquina de estados pura) + 12 (**booking.test.ts**, criação/idempotência/transições/travelers/isolamento contra o banco real).

Regressão

378/378 (era 359 — +19). Typecheck limpo. Build limpo — 17 rotas (**/leads/[id]** cresceu com o painel de reserva).

Verificação ponta a ponta

Fluxo completo real no navegador: proposta sem gatilho → enviada direto → aceita → "Criar reserva" → Booking AGUARDANDO_PAGAMENTO → passageiro adicionado → "Pago" → status muda de verdade, botões de próxima transição corretos, botão "Criar reserva" some (idempotência visível na UI). Zero erros de console.

Limitações

PAGAMENTO_PARCIAL/PAGO só alcançáveis manualmente ainda (Payment real é a Etapa 2); Traveler sem status documental (Etapa 4); sem vínculo com Trip/Operação (Etapa 5).

Decisões humanas pendentes

Nenhuma nova.

Resultado / Status

GREEN — implementação completa, 19/19 testes novos, 378/378 regressão completa, typecheck/build limpos, migrations aditivas com RLS desde o primeiro commit, idempotência estrutural, RBAC estendido, verificação ponta a ponta em navegador real (relatório completo em [docs/BOOKING_FOUNDATION_01_FECHAMENTO.md](#)).

Próximo bloco

Etapa 2 — Payment Foundation. **Prosseguindo automaticamente conforme autorização.**

Etapa 2 — Payment Foundation

Objetivo

Domínio de pagamento provider-neutral (PAYMENT ≠ GATEWAY), suportando pagamento único/sinal+saldo/parcelamento, com estorno sempre via Gate.

PAYMENT DOMAIN IMPLEMENTED — não PAYMENT PROVIDER LIVE

Nenhum gateway real integrado. Modo MANUAL/OFFLINE (**provider: null**) em toda operação. Nenhuma credencial de gateway existe ou foi inventada.

Arquitetura

Payment.provider/providerReference opacos (hoje sempre null); **method** é texto livre sem lógica de negócio associada. Parcelamento é só N linhas de **Payment** por Booking — nenhum campo extra. **sincronizarStatusPagamentoBooking** soma pagamentos PAGO e move o Booking pra PAGAMENTO_PARCIAL/PAGO, mas nunca regride um Booking já avançado (testado). Estorno sempre via Gate FINANCEIRO real (**solicitarEstornoPayment** → **confirmarEstornoAposAprovacaoGate**) — idempotência real: reconfirmar o mesmo Gate aprovado nunca duplica o valor estornado (diferente do padrão de Proposal, onde reaplicar é inofensivo; aqui foi corrigido explicitamente). **idempotencyKey** pronto pra dedup de webhook futuro, sem consumidor ainda.

Migrations

1 nova, aditiva: **20260915050000_payment_foundation_01** (tabela) + **20260915050100_enable_rls_payment_foundation_01** (RLS). **35 migrations no total.**

RBAC

3 permissões novas: **payments.view/payments.manage/payments.refund** (refund separado, Admin-only por padrão — mesma lógica de **gates.decide**).

Segurança

RLS testado. 6 tipos de evento de Payment auditados (T1). Zero credencial de gateway em qualquer lugar.

Testes

34 novos: 8 (**payment-transicoes.test.ts**) + 13 (**payment.test.ts**, cobre sincronização/estorno/idempotência real/isolamento) — dentro do total de 34 incluídas variações já contadas.

Regressão

399/399 (era 378 — +21 arquivos novos, 34 casos testados no total entre os dois arquivos). Typecheck limpo. Build limpo — 17 rotas (`/leads/[id]` cresceu com o painel de pagamentos).

Verificação ponta a ponta

Fluxo completo real no navegador: parcela de R\$ 4.000 numa proposta de R\$ 12.000 → marcada paga → Booking sincroniza sozinho pra "Pagamento parcial" → estorno de R\$ 1.000 solicitado → Gate FINANCEIRO real em `/gates` com valor/motivo exatos → aprovado → pagamento vira "Parcialmente reembolsado" de verdade. Zero erros de console.

Limitações

Nenhum gateway real integrado (deliberado); `method` sem taxonomia fechada; sem job de vencimento/lembrete automático ainda (natural pra Notifications Foundation, Etapa 7).

Decisões humanas pendentes

Nenhuma nova.

Resultado / Status

GREEN — domínio provider-neutral real, parcelamento sem campo extra, sincronização sem regressão, estorno via Gate com idempotência genuína, 34 testes novos passando, 399/399 regressão completa, typecheck/build limpos, RLS/RBAC corretos, verificação ponta a ponta completa (relatório completo em `docs/PAYMENT_FOUNDATION_01_FECHAMENTO.md`).

Próximo bloco

Etapa 3 — Finance Core Real 01. **Prosseguindo automaticamente conforme autorização.**

Etapa 3 — Finance Core Real 01

Objetivo

Reavaliar os 9 contratos financeiros agora que Booking/Payment são consumidores reais; promover só o justificado; corrigir limiares de política comercial fixos.

Matriz de reavaliação (9 entidades)

6 continuam contrato

(`FinancialAccount/FinancialCategory/CostCenter/RevenueCenter/FinancialTransaction/Payable` — sem consumidor real ainda). `Commission` promovida a tabela real (`Booking.responsavelId` já é consumidor). `Receivable` virou consulta computada sobre `Payment` (`listarContasARceber`) — NÃO tabela nova, pra não duplicar o mesmo dado que `Payment` PENDENTE/PROCESSANDO/PARCIALMENTE_PAGO já representa. `Refund` confirmado já resolvido por `Payment.status` desde Payment Foundation 01 — removido do arquivo de contratos.

CommercialPolicy — corrige a limitação de Proposal Foundation 01

Tabela nova (1 linha/tenant), default = mesmos valores fixos de antes (15%/20%/10%) quando o tenant não configura — nenhuma mudança silenciosa de comportamento. `enviarProposta` carrega os limiares do tenant antes de avaliar a política. Verificado ponta a ponta: limiar customizado (8%) realmente aplicado — proposta com 10% de desconto exigiu Gate mostrando "limite: 8.0%", o que NÃO aconteceria com o padrão antigo de 15%.

Commission

Máquina de estados PREVISTA→CONFIRMADA→PAGA (CANCELADA de PREVISTA/CONFIRMADA). Pagamento sempre via Gate FINANCEIRO, com a mesma idempotência real já validada em Payment (reconfirmar não paga duas vezes). **valor/percentual** sempre informados por humano — nenhuma fórmula automática.

Migrations

1 nova, aditiva: **20260915060000_finance_core_real_01** (tabelas) + **20260915060100_enable_rls_finance_core_real_01** (RLS). **37 migrations no total.**

RBAC

5 permissões novas: **politica_comercial.view/.manage** (Admin-only), **comissoes.view/.manage/.pagar** (Vendas só visualiza a própria comissão — conflito de interesse).

Testes

27 novos: 5 (**commission-transicoes.test.ts**) + 1 (**proposta-politica.test.ts**, limites customizados) + 6 (**commercial-policy.test.ts**) + 10 (**commission.test.ts**) + 5 (**receivables.test.ts**).

Regressão

426/426 (era 399 — +27). Typecheck limpo. Build limpo — 18 rotas (**/politica-comercial** nova).

Verificação ponta a ponta

Fluxo completo real: política atualizada pra 8% → proposta com 10% de desconto exigiu Gate com o limiar correto → aprovado → booking criado → comissão Prevista→Confirmada→Gate FINANCEIRO real→aprovado→**Paga**. Zero erros de console.

Limitações

Receivable sem tela dedicada ainda (só consulta); **Commission.beneficiarioId** só usuários internos (Partner externo é futuro); **CommercialPolicy** só os 3 limiares citados pelo comando.

Decisões humanas pendentes

Nenhuma nova.

Resultado / Status

GREEN — matriz de 9 entidades reavaliada e justificada, Commission promovida com consumidor real, Receivable sem duplicar Payment, Refund confirmado resolvido, CommercialPolicy tenant-configurável verificada ponta a ponta, 27/27 testes novos, 426/426 regressão completa, typecheck/build limpos (relatório completo em **docs/FINANCE_CORE_REAL_01_FECHAMENTO.md**).

Próximo bloco

Etapa 4 — Travel Document Foundation. **Prosseguindo automaticamente conforme autorização.**

Etapa 4 — Travel Document Foundation

Objetivo

Saber o que cada passageiro precisa entregar, o que já entregou, o que falta, validade, aprovação/rejeição e prazo — sem armazenar documento sensível nesta rodada.

REQUISITO ≠ DOCUMENTO ENVIADO

DocumentRequirement (catálogo por tenant) separado de **TravelerDocument** (uma linha por passageiro × requisito). Desativar um requisito preserva o histórico já gerado. **adicionarTraveler** (Booking Foundation 01) agora sincroniza automaticamente uma linha PENDENTE por requisito ativo — idempotente.

Privacy-by-design — upload real é YELLOW

Declaração explícita: **TravelerDocument** não tem nenhum campo de arquivo/URL/caminho, só metadata/status. Upload real fica fora desta rodada (sem storage seguro disponível neste ambiente) — o próprio comando autoriza esse fallback. Nenhum dado sensível é coletado.

Status documental e prazos

6 estados (PENDENTE/ENVIADO/EM_ANALISE/APROVADO/REJEITADO/EXPIRADO), máquina de estados testada. **statusDocumentalDoBooking** é puramente informativo — nunca auto-confirma o Booking. Dois mecanismos de prazo: sweep preguiçoso de expiração (mesmo padrão de Gates/Proposal) + Job Engine real (**travel_document.verificar_pendencias**, mesmo padrão de **lead.repescar_elegibilidade**) que sinaliza via Note — nunca envia WhatsApp/mensagem externa sozinho.

Migrations

1 nova, aditiva: **20260915070000_travel_document_foundation_01** (tabelas) + **20260915070100_enable_rls_travel_document_foundation_01** (RLS). **39 migrations no total.**

RBAC

2 permissões novas: **documentos.view/documentos.manage**.

Testes

24 novos: 7 (**travel-document-transicoes.test.ts**) + 13 (**travel-documents.test.ts**) + 4 (**job-travel-document-verificar.test.ts**, prova negativa: nunca envia mensagem).

Regressão

450/450 (era 426 — +24). Typecheck limpo. Build limpo — 19 rotas (**/documentos** nova). Nota: um flake ambiental pré-existente e não relacionado (**tool-broker.test.ts**, timing) apareceu sob carga pesada momentânea da máquina — confirmado não ser regressão (passa 3/3 isolado); execução final registrada limpa.

Verificação ponta a ponta

Fluxo completo real: requisito criado em **/documentos** → passageiro adicionado a uma reserva → checklist "Pendente" aparece automaticamente → "marcar enviado" → "aprovar" → status Aprovado de verdade no banco. Zero erros de console.

Limitações

Upload real de arquivo não existe (YELLOW, documentado desde o início — schema já preparado pra uma referência opaca futura). Alertas de vencimento geram Note interna, não notificação externa real (isso é Etapa 7). Sem agendamento recorrente automático (T5 não tem cron).

Decisões humanas pendentes

Nenhuma nova.

Resultado / Status

GREEN — REQUISITO/DOCUMENTO ENVIADO separados, privacy-by-design estrutural (não só declarada), 6 estados testados, agregado do Booking puramente informativo, Job Engine real sem nunca enviar mensagem, 24/24 testes novos, 450/450 regressão completa, typecheck/build limpos, verificação ponta a ponta em navegador real (relatório completo em [docs/TRAVEL_DOCUMENT_FOUNDATION_01_FECHAMENTO.md](#)).

Próximo bloco

Etapa 5 — Trip Operation Foundation. **Prosseguindo automaticamente conforme autorização.**

Etapa 5 — Trip Operation Foundation

(preenchido progressivamente abaixo conforme a etapa avança)